Санкт-Петербургский государственный университет

Чан Пэнсян

Отчет по производственной практике

Развитие специализированной системы запуска NICA-Scheduler эксперимента BM@N для распределенного запуска задач обработки на вычислительных платформах с различными типами систем планирования и требованиями к выполняемым задачам

Уровень образования: *Магистратура*

Направление *09.04.04 «Программная инженерия»*

Научный руководитель:

Доцент кафедры выислительной физики

Немнюгин Сергей Андреевич

Санкт-Петербург
2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3
1 Цель и задачи 3
2 Обзор 5
2.1 BMNROOT и его взаимосвязь с NICA-scheduler 5
2.2 Принцип работы NICA-scheduler 6
2.3 Спецификация описания задач на основе XML 7
2.4 Реализация функции автоматического создания выходного каталога и проверки ошибок при его отсутствии 9
Заключение 13
Список литературы 14

ВВЕДЕНИЕ

Ключевым этапом параллельной обработки данных в эксперименте BM@N является использование специализированной системы пакетной обработки для распределения, запуска и отслеживания выполнения задач обработки данных. BMNroot, как специализированная аналитическая структура данных для этого эксперимента, является основным инструментом, поддерживающим автономную обработку данных столкновений частиц, моделирование и визуализацию результатов. Весь анализ экспериментальных данных, включая их интерпретацию, реконструкцию и физический анализ, зависит от структуры BMNroot. Вычислительные кластеры, используемые в эксперименте, развертывают различные системы пакетной обработки, такие как Torque, SLURM, Condor, которые отличаются по количеству выполняемых задач, ограничениям на очередь задач и другим параметрам. Это приводит к таким проблемам, как сложность операций и недостаточная совместимость при межкластерной отправке и выполнении задач анализа данных на основе BMNroot.

NICA-Scheduler, разработанный специально для эксперимента BM@N (реализован на C++), играет ключевую роль моста между задачами анализа данных BMNroot и гетерогенными пакетными кластерами. Он не только упрощает процесс отправки задач в гетерогенной среде, адаптируется к индивидуальным потребностям пользователей в задачах, но и решает определенные проблемы, возникающие при обработке данных экспериментов со столкновениями частиц. Это обеспечивает эффективное и упорядоченное выполнение масштабных задач обработки данных на основе BMNroot, что делает его важной частью программного обеспечения для параллельной обработки данных в распределенной вычислительной архитектуре эксперимента.

1 Цель и задачи

Цель практики — дальнейшая разработка системы NICA-Scheduler. В рамках реализации проекта были поставлены следующие задачи:

1. Создавать все каталоги в пути к выходному файлу, если они не существуют (используя команду mkdir -p). Перед началом работы проверить, возникает ли ошибка, если каталог для выходных файлов не существует.
2. Добавить опцию extras=FILE_PATH в тег <get> для получения дополнительных файлов результатов из промежуточного хранилища. FILE_PATH может включать регулярные выражения.
3. Реализовать возможность принимать список файлов для обработки в качестве входного JSON-файла. Список должен задаваться новой опцией json_input тега <file>. Это аналогично опции file_input, но работает не с текстовым файлом, содержащим элементы на отдельных строках, а со списком файлов в формате JSON.
4. Проверить все опции слияния (merge) для объединения частичных файлов результатов. Реализовать автоматическое определение имени дерева (tree name), хранящегося в выходных файлах, для конструктора TChain вместо фиксированного значения ("bmndata").
5. Реализовать возможность добавлять тег <command> в стек заданий <job>, включающий набор макросов. В настоящее время тег <command> может быть только одной командой в задании, но он должен чередоваться с тегами <macro>.
6. Новые опции queuelimit=NNN и arraylimit=NNN для тега <run>, чтобы преодолеть ограничения на максимальное количество заданий, которые пользователь может отправить в очередь, и ограничение на размер массива подзадач соответственно. Это может быть разработано, например, путем создания основного shell-скрипта с циклом или путем реализации непрерывного мониторинга в фоновом процессе, который отслеживает текущие параметры выполнения пользовательских задач и запускает следующую порцию задач.

2 Обзор

2.1 BMNROOT и его взаимосвязь с NICA-scheduler

BmnRoot — это специализированный программный фреймворк, разработанный для эксперимента BM@N (Baryonic Matter at Nuclotron), который проводится на ускорительном комплексе NICA в Объединенном институте ядерных исследований (ОИЯИ). Являясь ядром цепочки обработки экспериментальных данных, BmnRoot тесно интегрирован в распределенную вычислительную среду, обеспечивая полный жизненный цикл данных: от моделирования отклика детекторов и реконструкции «сырых» данных до финального физического анализа.

BmnRoot имеет модульную иерархическую структуру, технологический стек которой выстраивается снизу вверх. На уровне базовых зависимостей (ROOT & FairSoft) лежит фреймворк ROOT, разработанный в ЦЕРНе, обеспечивающий хранение больших объемов данных (TTree), математические расчеты и визуализацию, а также интегрирован набор пакетов FairSoft, включающий компиляторы, физический движок Geant4 и другие базовые инструменты для научных вычислений. Промежуточный уровень (FairRoot), от которого наследует BmnRoot, разработан в Гейдельбергском университете и предоставляет универсальные механизмы управления потоком задач, систему управления объектами (иерархия задач) и эффективные интерфейсы ввода-вывода данных, что позволяет разделить алгоритмическую логику и низкоуровневую работу с данными. Наконец, уровень реализации приложения (специфические компоненты BmnRoot) является ключевым бизнес-уровнем фреймворка, который объединяет специализированные алгоритмы и модели, разработанные специально для детекторов BM@N, включая геометрические модели, алгоритмы оцифровки (digitization), трекинга (восстановления треков) и поиска вершин (vertex finding).

В число его основных функциональных модулей входят симуляция, реконструкция данных, калибровка и выравнивание, а также управление потоком

данных. Модуль симуляции использует движок Geant4 для моделирования методом Монте-Карло взаимодействия частиц с сильным магнитным полем и средой детектора. Модуль реконструкции данных преобразует исходные электронные сигналы, генерируемые детектором, в треки, импульсы и идентификацию частиц, обладающие физическими свойствами, что является ключевым звеном, связывающим экспериментальные измерения с физическим анализом. Модуль калибровки и выравнивания предоставляет механизмы оперативной коррекции для компенсации погрешностей измерений, вызванных геометрическим смещением детектора и факторами окружающей среды, обеспечивая физическую точность данных. Модуль управления потоком данных эффективно обрабатывает большие объемы экспериментальных двоичных данных и преобразует их в высокосжатые и удобные для анализа структурированные ROOT-файлы.

В задачах параллельной обработки больших данных эксперимента BM@N BmnRoot определяет логическое содержание вычислений (бизнес-логику), а NICA-Scheduler — способ выполнения вычислений (управление ресурсами). Взаимосвязь между ними прежде всего проявляется в среде исполнения задач: выполнение BmnRoot в значительной степени зависит от сложного программного окружения (например, конкретных версий GCC, CMake и путей к окружению FairSoft). Одна из ключевых функций NICA-Scheduler заключается в автоматизированной подготовке этого окружения для BmnRoot. Загружая файл cluster_config.sh или устанавливая переменные SIMPATH/FAIRROOTPATH в различных режимах (Local, Global, Container), планировщик гарантирует корректное выполнение макросов BmnRoot на гетерогенных кластерах (таких как Slurm, Torque, Condor) без каких-либо различий.

2.2 Принцип работы NICA-scheduler

NICA-Scheduler — это легковесный модуль управления задачами, специально разработанный для фреймворков на основе ROOT и FairRoot (таких как BmnRoot и MpdRoot). Его основная цель — абстрагировать сложность

нижележащих пакетных систем и предоставить исследователям унифицированный интерфейс для отправки задач.

Его основную логику работы можно разделить на четыре этапа. На первом этапе, этапе анализа и конфигурации, система считывает предоставленный пользователем XML-файл описания, извлекая имя задачи, путь к исполняемому макросу, списки входных и выходных файлов, а также требования к ресурсам, такие как CPU, память и очередь. Затем следует этап подготовки окружения, на котором планировщик, в зависимости от параметра config в теге <run>, автоматически загружает соответствующий Bash-скрипт для инициализации переменных окружения (например, SIMPATH). В режиме container он также отвечает за запуск указанного образа контейнера (например, Alma9 или CentOS7). Далее выполняется этап распределения задач и создания оберток: в локальном режиме (Local) используются многопоточность для параллельного запуска нескольких процессов ROOT на машине пользователя; в кластерном режиме (Global/Container) список <file> из XML разбивается на множество независимых заданий, и автоматически генерируются скрипты отправки (например, sbatch или qsub) для соответствующей пакетной системы (SLURM, SGE или Torque), что обеспечивает массовый параллелизм. Наконец, на этапе управления данными и мониторинга планировщик поддерживает автоматическую миграцию данных до и после выполнения задач (с помощью тегов <put> и <get>), а также выполняет слияние файлов (Merge) или очистку временных файлов (Clean) после завершения задач в соответствии с конфигурацией.

2.3 Спецификация описания задач на основе XML

NICA-Scheduler использует XML-формат для определения заданий, что обеспечивает высокую читаемость и расширяемость. Его структура следует иерархической логике «задание-макрос-файл-параметры запуска».

Тег	Описание функционала	Ключевые атрибуты

<job>	Определяет законченную единицу задания.	name (имя задания), batch (тип пакетной системы, по умолчанию slurm)
<macro>	Указывает исполняемый ROOT-макрос.	path (путь к файлу макроса), start_event, event_count
<file>	Определяет конкретные входные и выходные данные для этого макроса.	input (поддерживает регулярные выражения), output (путь вывода), merge (объединять ли)
<run>	Конфигурирует режим выполнения и квоты ресурсов.	mode (local/global/container), count (количество ядер), config (скрипт с переменными окружения)
<command>	Выполняет произвольные Shell-команды, не связанные с ROOT.	line (непосредственно командная строка)

Для обработки тысяч файлов XML-конфигурация поддерживает мощные встроенные переменные и теги для файловых операций. Что касается путевых переменных, в атрибуте output можно использовать такие переменные, как ${counter} (автоматический счётчик) и ${file_name} (исходное имя файла), чтобы гарантировать уникальность имён выходных файлов при параллельном

выполнении задач. В части миграции данных тег <put> отвечает за копирование данных перед началом вычислений из медленного хранилища (например, EOS) в быстрый временный каталог на вычислительном узле; тег <get> записывает результаты обратно в постоянное хранилище после завершения задачи, а тег <clean> автоматически удаляет вспомогательные файлы и логи, созданные в процессе выполнения.

2.4 Реализация функции автоматического создания выходного каталога и проверки ошибок при его отсутствии

Моделирование задач в экспериментах по физике высоких энергий (BM@N) является очень ресурсоемким. При запуске задачи система немедленно использует значительный объем памяти для загрузки сложных геометрических моделей детекторов и файлов карт магнитного поля, а также требует значительного времени CPU для вычислений по отслеживанию частиц. Поэтому мы добавили в NICA-Scheduler следующую функцию: Создавать все каталоги в пути к выходным файлам, если они не существуют (с использованием команды mkdir -p). Перед началом работы проверять, возникает ли ошибка, если каталог для выходных файлов отсутствует. Благодаря этому потенциальные ошибки, связанные с путями ввода-вывода, переносятся из ресурсоемкого этапа вычислений на этап начальной конфигурации, что позволяет эффективно избежать непроизводительных потерь вычислительных ресурсов из-за отсутствия каталогов.

В NICA-Scheduler принцип реализации функции автоматического создания выходных каталогов коренным образом отличается в зависимости от режима работы (Run Mode).

Local mode:

В локальном режиме (local mode) перед непосредственным запуском задачи или подзадачи система извлекает путь к каталогу из указанного выходного файла и выполняет команду `mkdir -p` для его создания, если он отсутствует. Если

создание каталога завершается ошибкой (например, из-за недостаточных прав или недопустимого пути), задача прерывается: генерируется сообщение об ошибке, временные скрипты удаляются, а семафор освобождается. Это позволяет избежать запуска ресурсоёмких вычислений при некорректно настроенном выводе. В режиме отладки фиксируется успешное создание каталога. Таким образом, проверка наличия выходного каталога переносится на этап до начала основных расчётов, что предотвращает непроизводительные потери вычислительных ресурсов.

Тестирование функции:

В XML (Рисунок 2.1) мы указываем несуществующий путь для вывода результатов и включаем наш режим debug.

```
<job>
 <macro path="$VMCWORKDIR/macro/run/run_sim_bmn.C">
  <file input="$VMCWORKDIR/macro/run/dC.04gev.mbias.100k.urqmd23.f14" output="$VMCWORKDIR/macro/run/nimadesada/sdada/eve
test.root" start_event="0" event_count="5"/>
 </macro>
 <run mode="local" count="1" config="~/bmnroot/build/config.sh"/>
</job>
```

Рисунок 2.1 – XML файл для тестирования

```
Apptainer> ./nica-scheduler $(pwd)/../nica_scheduler/examples/bmn_sim_local.xml -d
DEBUG nica-scheduler$ Local command = . ~/bmnroot/build/config.sh; root -b -q '$VMCWORKDIR/macro/run/run_sim_bmn.C("$VMC
WORKDIR/macro/run/dC.04gev.mbias.100k.urqmd23.f14", "$VMCWORKDIR/macro/run/nimadesada/sdada/evetest.root", 0, 5)'
DEBUG nica-scheduler$ Task 1: Successfully ensured directory (Local): $VMCWORKDIR/macro/run/nimadesada/sdada
nica-scheduler$ Task 1 is running...
input data - $VMCWORKDIR/macro/run/dC.04gev.mbias.100k.urqmd23.f14
output data - $VMCWORKDIR/macro/run/nimadesada/sdada/evetest.root

temp_1__1773699543.sh: line 2: /root/bmnroot/build/config.sh: No such file or directory
   ------------------------------------------------------------------
  | Welcome to ROOT 6.30/08                        https://root.cern |
  | (c) 1995-2024, The ROOT Team; conception: R. Brun, F. Rademakers |
  | Built for linuxx8664gcc on Dec 30 2025, 14:33:23                 |
  | From tags/v6-30-08@v6-30-08                                      |
  | With g++ (GCC) 14.3.0                                            |
  | Try '.help'/'.?', '.demo', '.license', '.credits', '.quit'/'.q'  |
   ------------------------------------------------------------------


Processing $VMCWORKDIR/macro/run/run_sim_bmn.C("$VMCWORKDIR/macro/run/dC.04gev.mbias.100k.urqmd23.f14", "$VMCWORKDIR/mac
ro/run/nimadesada/sdada/evetest.root", 0, 5)...
[INFO] Media file used: /opt/bmnroot/database/geo_db/storage/../../../../..//opt/bmnroot/database/geo_db/storage/media.g
eo
Info in <TGeoManager::TGeoManager>: Geometry Geometry, default geometry created
[INFO] FairRootFileSink initialized.
[INFO]  - bmnroot_0
[INFO]     - /opt/bmnroot/macro/run/nimadesada/sdada/evetest.root
```

Рисунок 2.2 – Результат после запуска NICA-Scheduler

Мы видим, что перед фактическим запуском вычислений выходной каталог был предварительно создан (Рисунок 2.2), что позволило избежать нерациональных затрат вычислительных ресурсов из-за отсутствия каталога.

В XML(Рисунок 2.3) мы указываем путь для вывода результатов, на который нет прав на запись, и включаем наш режим debug.

```
<job>
 <macro path="$VMCWORKDIR/macro/run/run_sim_bmn.C">
  <file input="$VMCWORKDIR/macro/run/dC.04gev.mbias.100k.urqmd23.f14" output="/sys/nicatest/evetest.root" start_event="0
" event_count="5"/>
 </macro>
 <run mode="local" count="1" config="~/bmnroot/build/config.sh"/>
</job>
```

Рисунок 2.3 – XML файл для тестирования

```
Apptainer> ./nica-scheduler $(pwd)/../nica_scheduler/examples/bmn_sim_local_test1.xml -d
DEBUG nica-scheduler$ Local command = . ~/bmnroot/build/config.sh; root -b -q '$VMCWORKDIR/macro/run/run_sim_bmn.C("$VMC
WORKDIR/macro/run/dC.04gev.mbias.100k.urqmd23.f14", "/sys/nicatest/evetest.root", 0, 5)'
mkdir: cannot create directory '/sys/nicatest': Permission denied
ERROR nica-scheduler$ Failed to create output directory for task 1: /sys/nicatest
nica-scheduler$ Local job has been finished
Apptainer>
```

Рисунок 2.4 – Результат после запуска NICA-Scheduler

Мы обнаружим, что программа немедленно перехватывает ошибку и останавливает задачу(Рисунок 2.4).

Global / Container Mode:

Поскольку в глобальном режиме и режиме контейнера генерируется пакетный скрипт (job_*.sh), который выполняется на вычислительном узле, необходимо напрямую внедрить команду `mkdir -p` в этот сгенерированный bash-скрипт. Это гарантирует, что во время фактического выполнения задачи требуемые каталоги будут точно созданы в файловой системе кластера (например, EOS или локальный временный диск).

Тестирование функции:

В XML (Рисунок 2.5) мы указываем запуск нашего NICA-Scheduler в глобальном режиме.

```
<job>
 <macro path="$VMCWORKDIR/macro/run/run_sim_bmn.C">
  <file input="$VMCWORKDIR/macro/run/dC.04gev.mbias.100k.urqmd23.f14" output="$VMCWORKDIR/macro/run/nimadesada/sdada/eve
test.root" start_event="0" event_count="5"/>
 </macro>
 <run mode="global" count="1" config="~/bmnroot/build/config.sh"/>
</job>
```

Рисунок 2.5 – XML файл для тестирования

```
element=${PutArray[${array_index}]}
if [ "${element}" != ">" ]
then
 # Check if PutElement contains "wait <file_path>"
 if [[ "${element}" == *"wait "* ]]; then
   # Extract file path after "wait "
   lock_file_path="${1#wait }"
   # Wait until file appears
   while [ ! -f "$lock_file_path" ]; do
    echo "DEBUG nica-scheduler: wait while putting file"
    sleep 5
   done
 else
  set -f # avoid globbing (expansion of *)
  # arr=(${element// / })
  # comm="${arr[0]} ${arr[1]} ${arr[2]}"
  # eval $comm
  echo "DEBUG nica-scheduler: PUT command = ${element}"
  eval ${element}
 fi
fi

element=${InputFileArray[${array_index}]}
export ROOT_HIST=0
comm="root -b -q '$VMCWORKDIR/macro/run/run_sim_bmn.C(\"${element}\""
element=${OutputFileArray[${array_index}]}
if [ "${element}" != ">" ]
then
  mkdir -p "$(dirname "${element}")"
 comm="${comm},\"${element}\""
fi
element=${StartEventArray[${array_index}]}
if [ "${element}" != ">" ]
then
 comm="${comm},${element}"
fi
element=${EventCountArray[${array_index}]}
if [ "${element}" != ">" ]
then
 comm="${comm},${element}"
fi
element=${AddArgsArray[${array_index}]}
if [ "${element}" != ">" ]
```

Рисунок 2.6 – Результат после запуска NICA-Scheduler

Мы обнаружили, что в сгенерированном скрипте успешно появилась команда mkdir -p "$(dirname "${element}")"(Рисунок 2.6). Это означает, что логика кода правильно внедрила инструкцию «предварительного создания каталога» в пакетное задание.

Заключение

Проведено глубокое техническое исследование и архитектурный анализ существующей системы: проанализирована и прояснена взаимосвязь между BmnRoot (уровень бизнес-логики) и NICA-Scheduler (уровень управления ресурсами). Изучено, как планировщик через переменные окружения (SIMPATH/FAIRROOTPATH) и контейнерный режим (Container Mode) обеспечивает унифицированную среду выполнения для BmnRoot. Системно исследованы спецификации описания задач на основе XML, выяснено, как планировщик преобразует определенную пользователем иерархическую логику «задача-макрос-файл» в исполняемые скрипты Shell и многопоточные задачи для различных пакетных систем (SLURM/SGE/Torque). На основе проведенных исследований в настоящее время реализована функция автоматического создания выходных каталогов.

В ходе дальнейшей работы планируется сосредоточиться на изучении основного C++ кода NICA-Scheduler с целью реализации таких функций, как разбор задач в формате JSON и динамическое управление лимитами на задания в кластере (queuelimit/arraylimit). Это позволит повысить уровень автоматизации обработки и гибкость планирования задач в распределенной среде.

Список литературы

1. Эксперимент BM@N NICA и программное обеспечение BMNRoot – URL: https://bmn.jinr.int/ (дата обращения: 2025-12-25).
2. Руководство пользователя NICA-Scheduler – URL: https://git.jinr.ru/nica-computing/nica_scheduler/-/blob/dev/README.md (дата обращения: 2026-01-03).
3. Примеры XML-описаний – URL: https://git.jinr.ru/nica-computing/nica_scheduler/-/tree/dev/examples (дата обращения: 2026-01-07).
4. Использование контейнера BMNRoot – URL: https://bmn.jinr.int/software-installation/ (дата обращения: 2026-01-10).
5. Руководство BMNRoot – URL: https://bmn.jinr.int/plug/BmnRoot.%20Start%20Guide.pdf (дата обращения: 2026-01-18).
6. Data analysis framework Root – URL: https://root.cern.ch/ (дата обращения: 2026-02-10).